

Predicting what makes a good wine from its chemical features

1. Introduction

Wine is among the most consumed alcoholic beverages in the world. Wine comes in numerous types, one of which is “Vinho Verde”, originating from Northern Portugal [1]. Wines can be categorized by multiple chemical metrics such as acidity, alcohol content or the amount of residual sugar. No two wines taste exactly the same, as their flavors are produced as a delicate combination of said metrics. Professional wine tasters have refined taste pallettes suited to detect even the slightest of variations between distinct vines. Due to the subtlety of their skill set, it may prove extremely challenging or even impossible for an algorithm to predict the quality of a wine given merely the initial parameters. Thus, machine learning algorithms may prove largely advantageous when deciding whether or not to buy a specific wine without prior knowledge of its taste.

This study will tackle the aforementioned problem by first formalizing it, covering the data briefly, discussing different machine learning methods in solving this problem, and finally utilizing said methods to the data and finally discussing the viability of these methods against each other.

2. Problem Formulation

The objective of this study is to predict the quality of a wine on a scale from 0 to 10 depending on certain characteristics of the wine. This problem is a regression problem as we are trying to predict the rating of a wine, and not classify the wines into different classes, such as poor, satisfactory, and great.

2.1. Data points

In this study, we used the Wine Quality repository from the UC Irvine Machine Learning Repository [1]. The repository includes two datasets about red and white Portuguese “Vinho Verde” -wine with a total of 4898 data points for white wine and 1599 data points for red wine. Each of the data points represents a single wine, and includes 12 variables, of which 11 are features.

2.2. Features

The features in this dataset consist of different chemical features of the wines. Some of these features are largely dependent on each other such as density and residual sugar, which the model then takes into account twice. In our initial model this was not taken into account, and all features were treated equally. Additionally none of the features were normalized nor standardized in our initial analysis.

No.	Feature	Range of data	Data type
0	Fixed Acidity	4.6-15.9	Continuous
1	Volatile Acidity	0.12-1.58	Continuous
2	Citric Acid	0-1	Continuous
3	Residual Sugar	0.9-15.5	Continuous
4	Chlorides	0.012-0.611	Continuous
5	Free Sulfur Dioxide	1-72	Continuous
6	Total Sulfur Dioxide	6-289	Continuous
7	Density	0.990-1.003	Continuous
8	pH	2.74-4.01	Continuous
9	Sulphates	0.33-2	Continuous
10	Alcohol	8.4-14.9	Continuous

Table 1: Features of the used data [1].

We did not find reliable data on the actual range of possible values for these chemical characteristics so we listed their range in the dataset [1].

2.3. Label

The label for this data set is the quality of the wine, rated by experts, as an integer ranging from 0 to 10. It is important to note, that even though the rating scale ranges from 0 to 10, the smallest rating in the dataset is 3 and the largest is 9, with the mean rating being around 5.7.

3. Methods

3.1. Preparing the Data

The data was first split into training, validation and testing data so that training data consists of 70% and validation and testing data both consist of 15% of the entire dataset. This split ensures the model has sufficient data for training while also allowing the model to be further developed by utilizing the validation data. The testing data will be utilized to determine the viability of the final model. No further data manipulation was required since the dataset had no missing values.

3.2. Model

A standard model of linear regression was utilized separately for both training datasets, red and white wine [2]. The model was chosen due to the numerical nature of both the features and labels making it effortless to utilize. The model also serves as a clear baseline for comparison with other models that will be utilized later in this study.

Mean squared error was utilized as the loss function for the model. The function was chosen since it is a standard baseline function for numerical results [2].

4. Results

4.1. Error in results

The validation root mean squared error of the linear regression model was determined to be 0.637 for the dataset of red wines and 0.763 for the dataset of white wines. Comparing these values to the mean ratings of red wines (5.64) and white wines (5.88) one can see that the RMS error is between 10-15%.

4.2. Visualization of results

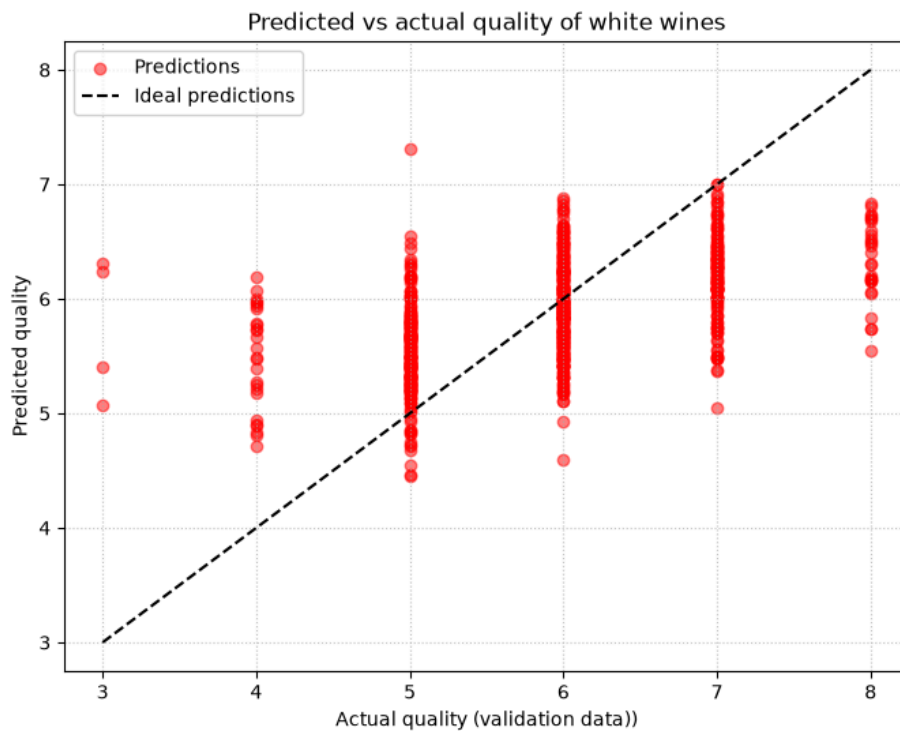


Figure 1: A visualization of the predictions of the model to the labels in white wine.

In the above figure, the x -axis represents the label (rating of the wine), the y -axis represents the prediction of the model, and the red dots represent individual data points, which are in this case white wines.

The dotted black line shows where the data points should lie, from where one can clearly see that the model performs poorly when the rating of the wine is low (3 or 4) or high (7 or 8). It is worthy to note that the regression model predicts on a continuous scale rather than a discrete integer scale.

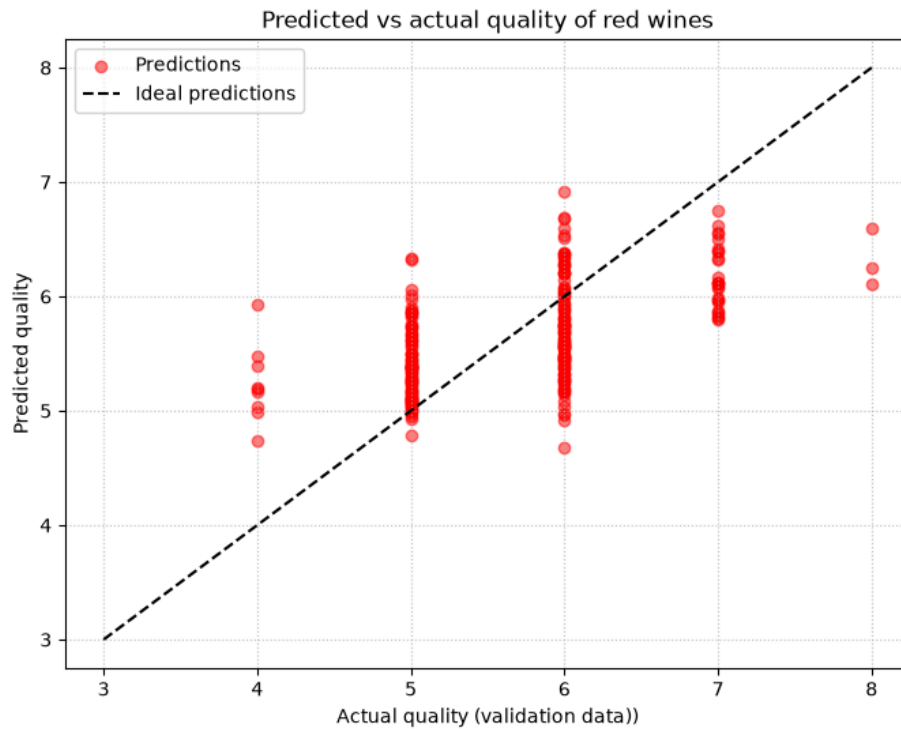


Figure 2: A visualization of the predictions of the model to the labels in red wine.

Similarly to Figure 1, the above figure visualizes the predictions of the model to the actual ratings of the red wines. The red wine model seems to behave similarly to the white wine model as the predictions are poor when the ratings differ from 5 or 6.

5. Conclusion

Using linear regression and a mean square error loss function, we were able to achieve a RMS error of under 15%. This is a solid baseline for this project, but can surely be improved upon when using more sophisticated methods. Since the dataset contained mostly wines with a quality of around 5, the model seems to struggle with predicting qualities at the extremes of the range. This can be clearly observed in figures Figure 1 and Figure 2.

Additionally, as discussed previously in this paper, the selection and processing of the features could be optimized, to further improve even this linear regression model.

6. Use of Artificial Intelligence

Artificial intelligence was used to configure our shared development environment. The environment consisted of VS Code as the IDE and GitHub as source control. For this, we utilized Aalto AI's "GPT 5.5" -model.

Bibliography

- [1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, Wine Quality. (2009). UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/dataset/186/wine+quality>

[2] A. Jung, Machine Learning: The Basics. Springer, Singapore, 2022.

Appendices

```
import numpy as np
import pandas as pd
import sklearn as scikit_learn
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Reading CSV's into pandas
datared = pd.read_csv('wine+quality/winequality-red.csv', sep=';')
datawhite = pd.read_csv('wine+quality/winequality-white.csv', sep=';')

# Extracting labels
y_red = datared["quality"]
y_white = datawhite["quality"]

# Amount of data points/labels
print("Amount of red wine data points:", y_red.count())
print("Amount of white wine data points:", y_white.count())

# Min and max of labels
print("Min and max rating of red wines:", y_red.min(), y_red.max())
print("Min and max rating of white wines:", y_white.min(), y_white.max())

# Mean of labels
print("Mean of rating for red wines:", y_red.mean())
print("Mean of rating for white wines:", y_white.mean())
print("\n")

# Extracting features
X_red = datared.drop(columns = "quality")
X_white = datawhite.drop(columns = "quality")

# Splitting red wine into train, test and validation sets (70/15/15)
X_red_train, X_red_left, y_red_train, y_red_left = train_test_split(X_red, y_red,
test_size=0.3, random_state=42)
X_red_val, X_red_test, y_red_val, y_red_test = train_test_split(X_red_left, y_red_left,
test_size=0.5, random_state=42)

# Same for white wine
X_white_train, X_white_left, y_white_train, y_white_left = train_test_split(X_white,
y_white, test_size=0.3, random_state=42)
X_white_val, X_white_test, y_white_val, y_white_test = train_test_split(X_white_left,
y_white_left, test_size=0.5, random_state=42)

print("RESULTS:")
# Red wine
reg1 = LinearRegression()
reg1.fit(X_red_train, y_red_train)
y_red_pred = reg1.predict(X_red_val)
tr_error_red = mean_squared_error(y_red_val, y_red_pred)
rms_red = np.sqrt(tr_error_red)
print("RMS error in red:", rms_red)
```

```

# White wine
reg2 = LinearRegression()
reg2.fit(X_white_train, y_white_train)
y_white_pred = reg2.predict(X_white_val)
tr_error_white = mean_squared_error(y_white_val, y_white_pred)
rms_white = np.sqrt(tr_error_white)
print("RMS error in white:", rms_white)

#Visualization

# White wine prediction figure
plt.figure(figsize=(8, 6))

# Predictions as a scatter plot
plt.scatter(y_white_val, y_white_pred, alpha=0.5, color='red', label='Predictions')

# Target predictions
plt.plot([3, 8], [3, 8], color='black', linestyle='--', label='Ideal predictions')

plt.title('Predicted vs actual quality of white wines')
plt.xlabel('Actual quality (validation data)')
plt.ylabel('Predicted quality')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)

plt.savefig("white_wine_pred.png")

# Red wine prediction figure
plt.figure(figsize=(8, 6))

# Predictions as a scatter plot
plt.scatter(y_red_val, y_red_pred, alpha=0.5, color='red', label='Predictions')

# Target predictions
plt.plot([3, 8], [3, 8], color='black', linestyle='--', label='Ideal predictions')

plt.title('Predicted vs actual quality of red wines')
plt.xlabel('Actual quality (validation data)')
plt.ylabel('Predicted quality')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)

plt.savefig("red_wine_pred.png")

```